

CS543/ECE549 Assignment 1

Name: Rojas Pelliccia, Máximo Gustavo

NetId: mgr9

Parts 1 and 2: Implementation Description

For all the following results, I have used NCC as a metric, but I have as well implemented SSD. There are configuration files to quickly change parameters if desired.

- **Part 1A (Basic Alignment) Description:**

Firstly, I separate the three channels from the image into unique numpy arrays to work on them individually. I crop and make them all the same size to work on them. Then, I just iterate through the values of the window, shifting the channels and evaluating them with the chosen metric. While iteration, the best offset and score for the metric are stored and when the iteration ends, they are used to shift the channel. Finally, the three channels are joined together with the blue one as the absolute one.

- **Part 1B (Multiscale Alignment) Description:**

Just the same as in Part 1A, the implementation starts with separating the channels, and a big difference, is that the hi-res images are on a 16bit scale and not 8bit. So, it is checked if the image being processed is a 16bit or not. In case it is, it is transformed into 8bit and then the alignment starts. The implementation of the recursive pyramid is as follows: downsample the image -> function call, until it reaches base case which is height = 0. Once it reaches to the bottom of the pyramid (or top) it starts to search the best alignments on the low resolution image, and when it finds it, it returns the offset to the previous recursive call so height = 1 can use that displacement to search for a better one. It is important to note that when upsampling the image, the offset also need to be upsampled by the same factor as well. This is cyclically done until the channels in the original resolution are aligned, for 2 channels (red and green, blue's position remains the same) and then they are joined together.

- **Part 2 (Fourier Alignment) Description:**

The main idea of the Fourier Alignment is the same as Part 1, but instead of searching the optimum offset with NCC or SSD the algorithm uses fourier transforms. Firstly, we calculate the frequency of the image and then we calculate the product of both to see how well do they align.

Parts 1 and 2: Channel Offsets

Offsets of each channel to the <Blue> channel using NCC:

Low-res Image	Basic Alignment (1A)		Fourier-based Alignment (2)	
	<Green> (h,w) offset	<Red> (h,w) offset	<Green> (h,w) offset	<Red> (h,w) offset
00125v.jpg	(6, -1)	(10, -2)	(6, 2)	(11, 1)
00149v.jpg	(4, 1)	(9, 1)	(5, 3)	(10, 3)
00153v.jpg	(7, 1)	(14, 2)	(8, 3)	(15, 4)
00351v.jpg	(4, 0)	(13, -1)	(5, 0)	(14, 1)
00398v.jpg	(6, -1)	(11, -2)	(6, 3)	(12, 5)
01112v.jpg	(0, 1)	(3, 2)	(1, 1)	(6, 2)
High-res Image	Multiscale Alignment (1B)		Fourier-based Alignment (2)	
	<Green> (h,w) offset	<Red> (h,w) offset	<Green> (h,w) offset	<Red> (h,w) offset
01047u.tif	(24, 20)	(72, 33)	(26, 20)	(71, 34)
01657u.tif	(47, 9)	(115, 10)	(57, 10)	(120, 14)
01861a.tif	(71, 38)	(148, 63)	(70, 38)	(146, 63)

Parts 1 and 2: Runtime Comparison and Discussion

Report the running time in terms of seconds (s).

Image	Basic (1A)	Multiscale (1B)	Fourier-based (2)
00125v.jpg	11.38	1.24	0.22
00149v.jpg	6.69	1.07	0.21
00153v.jpg	5.55	1.02	0.21
00351v.jpg	10.72	1.16	0.22
00398v.jpg	11.84	1.06	0.21
01112v.jpg	10.33	0.88	0.32
01047u.tif	390.10	98.83	13.61
01657u.tif	445.80	111.68	13.03
01861a.tif	395.08	132.22	14.72

The improvement between the basic alignment and the other two is substantial. We can see that for the .jpg images, the time is reduced almost in one order of magnitude for the multiscale and two for the Fourier based alignment. For the multiscale solution, I've downsampled the images twice (in my implementation, in the configuration file, height=2) and we can see this greatly increased the performance. Although the pyramid alignment performed well, the fourier alignment outperforms it.

For the .tif images, again, the performance jump between the three solutions can be clearly seen. The multiscale alignment, now with a height of 6 and crop size of 175px, took 3 times less as the basic alignment, while the Fourier alignment took an entire order of magnitude less.

From these results, the Fourier alignment seems the best performance wise, especially for the high-resolution images, where also the channel alignment is also very good.

Parts 1 and 2: Output Visualizations

Crop amount in each photo:

1A - Low-resolution images: 5

1B - High-resolution images: 175

2 - Low-resolution images: 5

2 - High-resolution images: 175

1A: Basic Alignment

00125v.jpg



00149v.jpg



00153v.jpg



00351v.jpg



00398v.jpg



01112v.jpg



1B: Multiscale Alignment
01047u.tif



01657u.tif



01861a.tif



2: Fourier-based Alignment

00125v.jpg



00149v.jpg



00153v.jpg



00351v.jpg



00398v.jpg



01112v.jpg



01047u.tif



01657u.tif



01861a.tif



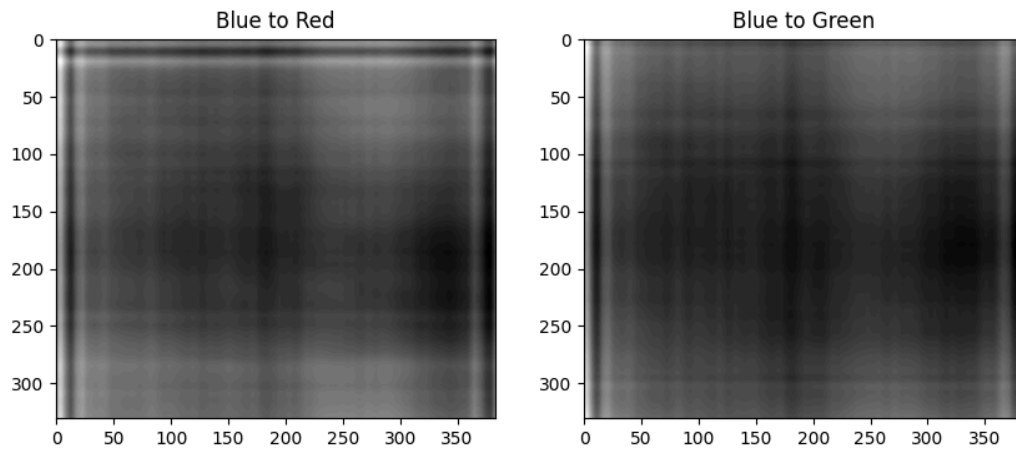
2: Inverse Fourier Transform Visualizations

00153v.jpg

B to G without processing

B to R without processing

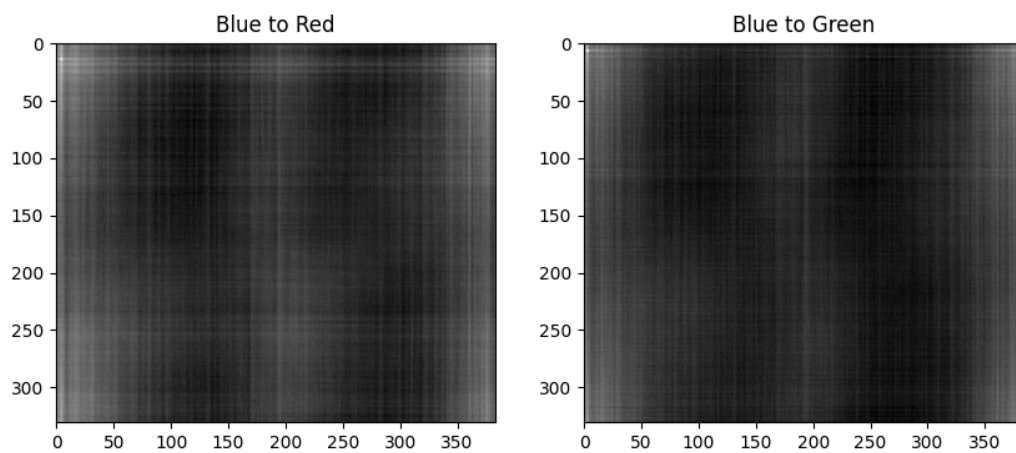
Without Processing



B to G with processing

B to R with processing

With Processing

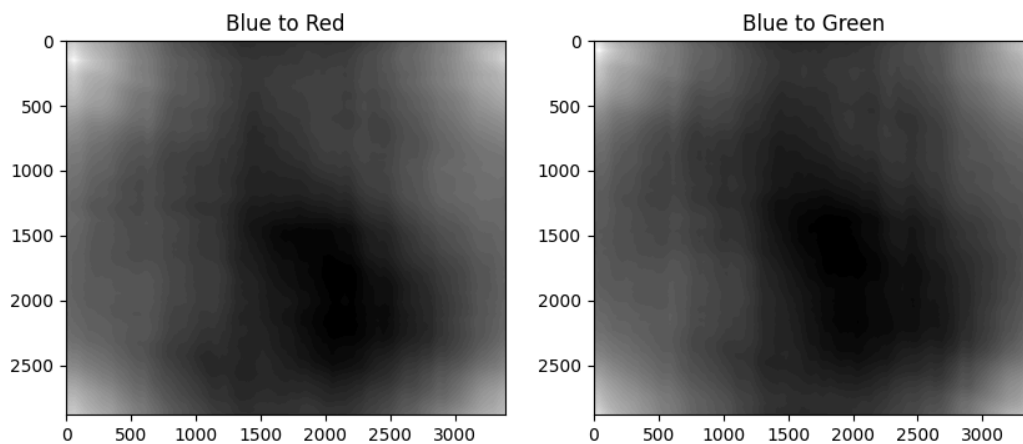


01861a.tif

B to G without processing

B to R without processing

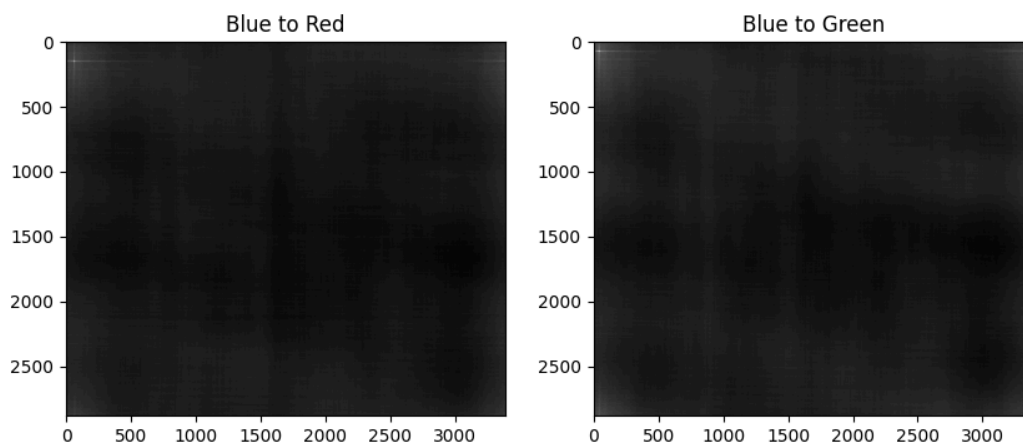
Without Processing



B to G with processing

B to R with processing

With Processing



Parts 1 and 2: Any additional discussion, bonus Improvements

When approaching part 1b, I had a very difficult time trying to align the images correctly. Nothing I tried worked for making the image align correctly. Then, at class, I've asked classmates about it, and they told me the problem was probably not with my code, but with the format of the images itself. Then I realised that the images were not in an 8bit format, which made me progress through the assignment.

Part 3: Hybrid images implementation discussion

For this part, the implementation was rather trivial as the only thing to do is implement the functions to apply a low-pass and high-pass filter to each of the images. The most difficult part was find good values of sigma to use when applying the filters. During my trial and error attempts on generating the hybrids, I realised that the sigma value for the high-pass filter should be bigger than the sigma used on the low-pass filter. Finally, for the generation of the images shown below, I've used different values for all images while maintaining the same viewing distance. It is important to clarify that these values are the best I have found the with the experiment I did, which is looking at my computer's screen from up close and then a couple of feet away. As seen in class, the optimal sigma depends on the distance to and aperture of the image.

Part 3: Hybrid images results

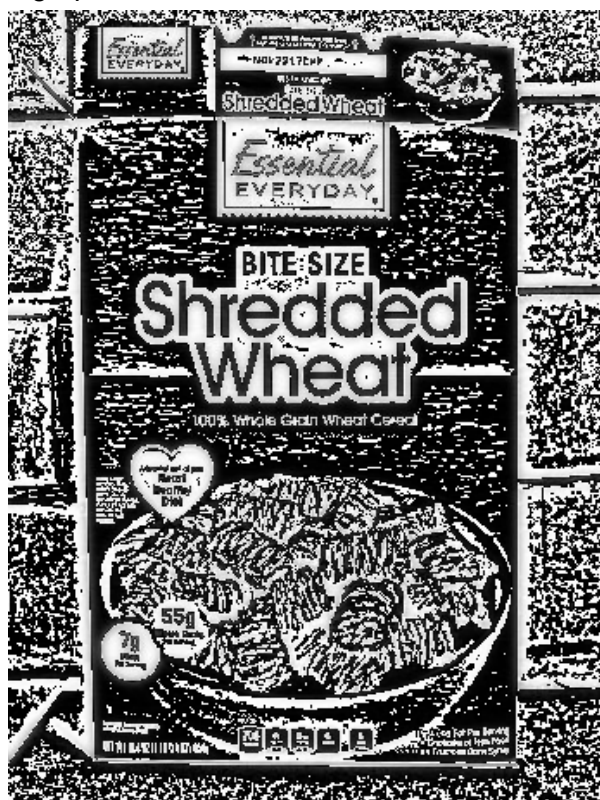
Example 1:

Input images:



Filtered input images:

High-pass filtered:



Low-pass filtered:



Hybrid image:

High-resolution:



Low-resolution:



Sigmas used:

High-pass filter sigma: 3

Low-pass filter sigma: 1

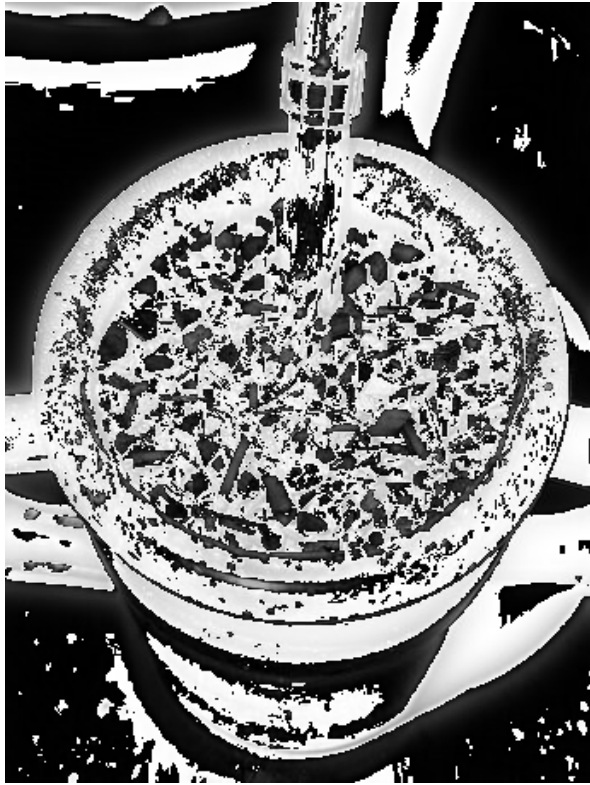
Example 2:

Input images:



Filtered Images:

High-pass filtered:



Low-pass filtered:



Hybrid images:
High-resolution:



Low-resolution:



Sigmas used:
High-pass filter sigma: 10
Low-pass filter sigma: 3

Example 3:

Input Images:



Filtered Images:

High-pass filtered:



Low-pass filtered:



Hybrid images:
High-resolution:



Low-resolution:



Sigmas used:
High-pass filter sigma: 15
Low-pass filter sigma: 8

Part 3: Results discussion, bonus improvements

The gotten results with the developed implementation were satisfactory. In some cases, the high-pass filter developed some very dark or white spots on the image which can be seen at a high-resolution adding noise and lowering the quality of the image produced. This was one of the biggest setbacks as a lot of trial and error was needed to find good pairs of sigmas since I wasn't able to fix it.

When analysing the results, I first came to the conclusion that images that have a high amount of details work better when applied a high-pass filter and the other way round for images with a low amount of details. This can be seen in the image of the mate and the paella, where the mate has lots of contour lines, edges and different shades of green while the paella is mostly uniform, except for the almost white-looking shrimps. Then, my first conclusion was that it is desirable to pair up an image with a low amount of details with one that has a high amount of detail, just as the leopard and elephant example in class. But after trial and error and applying working with different types of sigmas, I realized that this was not truly the case, or it was not an absolute certainty. I tried applying the filters to both, the detergent and milk jug, and to my surprise, the less detailed one (the milk jug) worked best when filtered with the high-pass filter.